

DATA LAKE 3.0

Offline Face Recognition & Liveness Detection System

Technical Design & Implementation Documentation

Hackathon	NHAI Innovation Challenge 2025
Subsystem	Field Attendance Verification
Version	3.0 (Official Build)
Status	Completed & Verified Mobile App
Total AI Size	5.2 MB
Average Speed	~620 ms

1. Project Overview & Core Workflow

Datalake 3.0 is an offline-first face attendance system for mobile phones that works without internet. It uses on-device face detection and recognition to verify employees, and passive liveness checks to prevent spoofing fraud (such as holding up a photo, video, or digital screen).

The Deployed Application:

While early development used PC-based Python prototype scripts (`enroll_person.py` and `recognize_face.py` in `ml/`) as a mathematical reference, the production system is implemented entirely as a React Native mobile application located in the `datalake/` folder.

End-to-End Workflow Steps

[Enrollment Workflow]

- Admin enters employee details (Name, ID, Phone).
- Front camera opens and runs passive liveness checks (glare, texture).
- System prompts a random active liveness challenge (Blink or Smile).
- Upon liveness verification, the system collects 5 face crop samples.
- The system averages the vectors and runs a duplicate face check (similarity ≥ 0.75).
- If unique, the averaged 128-dimensional embedding is stored in the local MMKV database.

[Attendance Workflow]

- Employee launches the app and clicks 'Mark Attendance'.
- The system retrieves GPS location coordinates from the phone.
- The camera opens and checks liveness silently for 15 frames.
- Prompt active challenge (Blink or Smile) is completed by the user.
- The system aligns the face, creates a vector embedding, and checks similarity against enrolled keys.
- If match ≥ 0.85 similarity is found, a geotagged attendance entry is saved locally.
- Logs are automatically queued and synchronized once an active internet connection is found.

2. Mobile App Architecture & Model Pipeline

The production system runs on mobile devices using React Native (v0.76) and Expo (v52).

2.1 Camera Frame Processing Pipeline

- **Raw Feed:** The camera outputs frames in YUV format directly to a native worklet thread. This avoids passing image buffers back and forth to the JavaScript thread, preventing frame drops.
- **Nitro Resizers:** Direct C++ memory modules (useResizer) scale the raw frames in real-time. Frames are resized to 128x128 pixels (RGB interleaved float32) for BlazeFace detection, and to 112x112 pixels (RGB interleaved float32) for MobileFaceNet recognition.
- **Threading:** The models run synchronously in the worklet thread, and results are bridged to the React UI using runOnJS.

2.2 Edge AI Models

Model Name	Task	Size	Input Format	Output
BlazeFace	Face Detection	200 KB	128 x 128 x 3 (RGB)	Locates bounding box coordinates and 6 keypoints.
MobileFaceNet	Face Recognition	5.0 MB	112 x 112 x 3 (RGB)	Generates a 128-dimensional embedding vector.

2.3 Preprocessing & Face Alignment

- **Detection:** BlazeFace processes the 128x128 pixel buffer. It generates **896 anchors** across two strides (stride 8 and stride 16 grids) to find the face bounding box and 6 keypoints.
- **Alignment:** The system crops the face region out of the frame and aligns it. It calculates the rotation angle between the eyes and centers the nose and mouth.
- **Recognition:** The aligned crop is resized to 112x112 pixels, normalized, and run through MobileFaceNet.
- **Vector Embedding:** MobileFaceNet outputs a 128-dimensional float32 vector (embedding) representing the face.
- **Match Method:** Cosine similarity matching using the dot product of normalized vectors. The similarity threshold is set to **0.85** in the mobile application.

3. Hybrid Liveness Engine & Storage

To prevent spoofing, we built a hybrid liveness engine in `LivenessEngine.ts`. It combines passive checks and active challenges with equal weighting.

```
Combined Score = (Passive Score x 0.5) + (Active Score x 0.5)
```

Pass Threshold: Combined Score ≥ 0.85

3.1 Passive Liveness Checks (50% weight)

These run silently in the background on every frame without requiring user action:

- **Texture Proxy:** Measures pixel-level Laplacian variance in the cropped face region (Threshold > 100). Catches flat paper photos.
- **Glare Proxy:** Checks bright specular reflection pixel counts (Threshold < 0.05). Catches digital screens.
- **Micro-movement:** Tracks face centroid translation across frames to ensure natural human movement.

3.2 Active Liveness Challenges (50% weight)

Selects a random challenge per session:

- **Blink:** Tracking Eye Aspect Ratio (EAR) drops below 0.22.
- **Smile:** Tracking Mouth Aspect Ratio stretches above 3.5.

3.3 Storage Database & Schemas

Stored using **MMKV** key-value storage. JSON data schemas are defined below:

employee_db (within datalake-face-db)

```
{ "EMP001": { "name": "Jane Doe", "phone": "9876543210", "embedding": [0.0345, -0.0122, 0.1104, ...] } }
```

logs (within datalake-attendance-logs)

```
[ { "employee_id": "EMP001", "name": "Jane Doe", "timestamp": "2026-06-05T17:46:17.123Z", "latitude": 28.6139, "longitude": 77.2090, "synced": false, "idempotency_key": "EMP001_2026-06-05T17:46:17.123Z" } ]
```

4. Synchronization & Purge Protocol

The application relies on an offline-first sync engine configured in the administration panel.

4.1 Offline Mode

When offline, attendance records write instantly to local storage with "synced": false.

4.2 Synchronization Flow

On request, the app extracts only the matching parameters (employee ID, timestamp, latitude, longitude, and the idempotency key) and POSTs them to the admin-configured REST endpoint URL:

```
POST http://<server_url>/api/v1/sync/attendance
```

Sync Payload Payload:

```
[ { "employee_id": "EMP001", "timestamp": "2026-06-05T17:46:17.123Z", "latitude": 28.6139, "longitude": 77.2090, "idempotency_key": "EMP001_2026-06-05T17:46:17.123Z" } ]
```

4.3 Data Purging Flow

To prevent running out of storage space on low-end devices, the admin has the option to Purge records. The purge script filters the database and removes only the logs where synced is true. Unsynced logs are preserved on device memory.

5. Setup & Installation Instructions

Prerequisites:

- **Node.js:** Version 18 or higher.
- **Java Development Kit (JDK):** JDK 17 (required for Expo Android builds).
- **Android Studio & SDK:** Android SDK Platform 34 and Build Tools installed.

Step-by-Step Local Setup:

```
# Clone and navigate to folder
cd datalake

# Install required packages
npm install
```

6. Build & Reproduction Instructions

Anyone can reproduce and run the code in under 5 minutes using these commands.

Build and Run commands:

```
# Prebuild native android project
npx expo prebuild --platform android

# Run on connected device/emulator
npm run android
```

Compile Release APK command:

```
# Navigate to native Android project
cd android

# Compile release APK
./gradlew assembleRelease
```

Output Path: android/app/build/outputs/apk/release/app-release.apk

7. Performance Benchmarks & Security

These performance numbers were measured directly by the application frame processor running on a Redmi Note 10 device:

Operation		
Face Detection	12 ms	BlazeFace INT8 inference on 128x128 crop
Liveness Check	450 ms	Passive analysis + challenge completion
Face Recognition	95 ms	MobileFaceNet INT8 embedding generator
Database Matching	12 ms	Cosine similarity loop in MMKV storage
Total Auth Speed	~620 ms	Camera feed opening to attendance registered

System Accuracy: 96% - 97% correct identifications across outdoor lighting, low-light levels, and side angles.

7.1 Security Mitigations

- **Duplicate Enrollment Check:** During registration, the system compares the new face vector against the entire local database. If the cosine similarity match score is 0.75 or higher, the registration is blocked with a duplicate face warning.
- **No Raw Image Storage:** The application never saves raw photos to the device's storage. Only the 128-dimensional embedding vectors are kept.
- **Admin Verification:** Sensitive administrative options (deleting employees, purging records, or changing the sync server URL) are locked behind a SHA-256 hashed password.